```
print(f"[SERVER] Listening on {host}:{port}...")
while True:
    client_socket, client_address = server_socket.accept()
    client_thread = threading.Thread(target=handle_
client, args=(client_socket, client_address)
)
client_thread.start()

def start_client(server_host="127.0.0.1",
server_port=5000):
client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
client_socket.connect((server_host, server_port))
print(f"[CLIENT] Connected to server {server_host}:
{server_port}")
try:
    while True:
        msg = input("Enter message (or 'quit' to exit): ")
        if msg.lower() == "quit":
            break
        client_socket.sendall(msg.encode())
        response = client_socket.recv(1024).decode()
        print(f"[SERVER RESPONSE] {response}")
finally:
    client_socket.close()
    print("[CLIENT] Disconnected")
if __name__ == "__main__":
    import sys
    if len(sys.argv) > 1 and sys.argv[1] == "server":
        start_server()
    else:
        start_client()
```

23-9-26

Ex: 12 a Echo client server using TCP/UDP sockets

AIM:

To implement echo client server using TCP/UDP sockets

TCP Echo Client - Server Program

```
import socket
import threading

# server code
def handle_client (client_socket, client_address):
    print (f"[+] New connection from {client_address}")
    while True:
        try:
            msg = client_socket.recv(1024).decode()
            if not msg:
                break
            Print(f"[Client {client_address}] {msg}")
            client_socket.sendall (f"server received: {msg}"
            .encode())
        except ConnectionResetError:
            break
    Print(f"[-] Connection closed {client_address}")
    client_socket.close()
def start_server (host="127.0.0.1", port=5000):
    server_socket = socket.socket (socket.AF_INET,
    socket.SOCK_STREAM)
    server_socket.bind ((host, port))
    server_socket.listen(5)
```

```
end = time.time()
rtt = (end - start) * 1000
print(f"Reply from {server_host}: {len(data.decode())} b" f"RTT = {rtt:.2f} ms")
except socket.timeout:
    print(f"Request {i} timed out")
client_socket.close()

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "s":
        udp_ping_server()
    else:
        udp_ping_client()
```

TCP Echo client server output:

```
[SERVER] Listening on 127.0.0.1:5000...
[+] New connection from ('127.0.0.1', 54321)
[Client ('127.0.0.1', 54321)] Hello
[Client ('127.0.0.1', 54321)] Computer Networks
[Client ('127.0.0.1', 54321)] TCP Socket Programming
```

Client Terminal output:

```
[CLIENT] Connected to server 127.0.0.1:5000
Enter message (or 'quit' to exit): Hello
[SERVER RESPONSE] Server received: Hello
Enter message (or 'quit' to exit): Computer Networks
[SERVER RESPONSE] Server received: Computer Networks
Enter message (or 'quit' to exit): TCP Socket Programming
[SERVER RESPONSE] Server received: TCP Socket Programming
Enter message (or 'quit' to exit): quit
[CLIENT] Disconnected
```

UDP Echo client server Program

```
import socket
import time
import sys
def udp_ping_server (host = "127.0.0.1", port = 12000):
    server_socket = socket.socket (socket.AF_INET,
socket.SOCK_DGRAM)
    server_socket.bind ((host, port))
    print (f"[Server] listening on {host}:{port}")
    while True:
        msg, client_address = server_socket.recvfrom(1024)
        print (f"[SERVER] Received '{msg.decode()}'"
            f" from {client_address}")
        server_socket.sendto (msg, client_address)
def udp_ping_client (
        server_host = "127.0.0.1",
        server_port = 12000,
        count = 5
    ):
    client_socket = socket.socket (socket.AF_INET,
socket.SOCK_DGRAM)
    client_socket.settimeout (1)
    for i in range (1, count + 1):
        msg = f"ping {i} {time.time()}"
        start = time.time()
        client_socket.sendto (msg.encode(),
        server_host, server_port))
    try:
        data, _ = client_socket.recvfrom (1024)
```

UDP ping output:

[SERVER] Listening on 127.0.0.1:12000
[SERVER] Received 'Ping 1 172836...' from ('127.0.0.1', ...
[SERVER] Received 'Ping 2 172836...' from ('127.0.0.1', ...
[SERVER] Received 'Ping 3 172836...' from ('127.0.0.1', ...
[SERVER] Received 'Ping 4 172836...' from ('127.0.0.1', ...
[SERVER] Received 'Ping 5 172836...' from ('127.0.0.1', ...

Client Terminal output:

Reply from 127.0.0.1:12000 | Ping 1 172836... | RTT=0.xx ms
Reply from 127.0.0.1:12000 | Ping 2 172836... | RTT=0.21 ms
Reply from 127.0.0.1:12000 | Ping 3 172836... | RTT=0.19 ms
Reply from 127.0.0.1:12000 | Ping 4 172836... | RTT=0.20 ms
Reply from 127.0.0.1:12000 | Ping 5 172836... | RTT=0.17 ms

RESULT:

Thus, the TCP and UDP echo client server programs were implemented successfully using Python Socket Programming.

23-9-24 Ex. 12 b Implement chat client server using TCP/UDP sockets

AIM:

To implement chat client server using TCP/UDP sockets.

Procedure:

1) Create server pgm and client pgm.
2) Import the required socket library.
3) In server, create a socket and bind it to an IP address and port no.
4) Make the server listen to client request.
5) Start client & and connect it to server.
6) Enter message from client
7) Send msg to server
8) The server receives the msg and sends same msg back.
9) The client receives and displays the echoed msg.
10) close socket connection.

b) Client server using socket pgm:

1) Create server and client pgm using socket library
2) Start server and bind it to IP address & port
3) Make server wait for client connection.
4) Start client and connect it to server
5) Enter msg in client
6) Send msg in server.
7) The server receives and displays msg.
8) The server sends a reply back to client
9) Continue sending and receiving message to perform chat.
10) close connection after chat is completed.

RESULT:

Thus the chat client server is implemented using TCP/UDP sockets.